

Enunciado:

Num cenário em que vários Large Language Models comerciais — como ChatGPT, Claude e Llama — oferecem funcionalidades e resultados cada vez mais parecidos, explique por que essa convergência torna a segurança mais frágil, focando nos chamados ataques de extração de modelo (model extraction attacks). Descreva como esse tipo de ataque funciona e por que a semelhança entre modelos aumenta a chance de alguém conseguir roubar propriedade intelectual.

Resposta:

Quando vários modelos comerciais passam a se comportar de maneira parecida, isso facilita bastante quem quer copiar ou imitar um modelo protegido. O ataque de extração basicamente consiste em fazer um grande número de requisições à API do modelo alvo (o “victim model”) com prompts bem pensados, coletar as respostas e usar esse conjunto de entrada/saída para treinar um modelo próprio — o chamado modelo pirata. Em termos práticos, o atacante não precisa acessar os pesos nem a arquitetura original: basta reproduzir o comportamento através de muitos exemplos. A razão pela qual a similaridade entre modelos piora o problema é simples: técnicas de sondagem e conjuntos de prompts que funcionam num modelo tendem a funcionar também em outros semelhantes. Isso significa que o atacante pode aplicar a mesma estratégia em vários provedores, reaproveitando prompts, padrões de queries e heurísticas de extração, o que reduz tempo e custo do ataque. Em vez de gastar milhões para treinar um modelo do zero, basta pagar pelas chamadas de API para obter respostas que servem como “rótulos” de alta qualidade para treinar um substituto com fidelidade elevada. Em resumo, quanto mais homogêneos os modelos, maior a transferência das técnicas de ataque e mais viável — técnica e economicamente — se torna a cópia do comportamento e das capacidades que são, na prática, propriedade intelectual dos provedores.

Enunciado:

Explique por que a implementação de soluções de segurança em modelos de linguagem (LLMs) costuma aumentar o custo de desenvolvimento e operação desses sistemas.

Resposta:

A implementação de soluções de segurança em modelos de linguagem aumenta o custo porque exige ajustes adicionais na arquitetura e no treinamento do modelo. Algumas medidas são relativamente simples, como o uso de marcas d'água ou filtros externos, mas soluções mais robustas demandam a inclusão de novos parâmetros, pesos e até camadas extras na rede neural, que precisam ser retreinados para incorporar comportamentos seguros. Além disso, mecanismos de segurança mais avançados requerem monitoramento e checagem constante das respostas do modelo, o que aumenta o consumo computacional e, consequentemente, os custos de processamento e manutenção. Outro fator que eleva os custos é a necessidade de empregar métodos de aprendizado por reforço com feedback humano e técnicas de filtragem e verificação de dados para garantir que o modelo aprenda comportamentos éticos e evite respostas ofensivas/ruins. Esse processo demanda uma equipe especializada, tempo de anotação e grande volume de exemplos supervisionados. Em

geral, novas medidas de segurança envolvem verificações a mais durante a execução do sistema, uso de outros modelos de linguagem para analisar inputs/outputs, filtros textuais, melhorias de prompts, além de mais dados e etapas de treinamento (seja fine-tuning ou aprendizado por reforço). Todas essas medidas envolvem, em maior ou menor quantidade, mais esforço, tempo e dinheiro para implementação e testes.

Enunciado:

Com o avanço dos modelos de linguagem, atacantes têm explorado suas vulnerabilidades para realizar ações maliciosas, como o roubo de dados ou a execução de comandos indevidos. Explique quais são as principais estratégias de ataque utilizadas contra LLMs e quais mecanismos podem ser aplicados para se defender dessas ameaças.

Resposta:

Os ataques a LLMs são feitos de forma geral via APIs, em que os atacantes fazem diversas requisições e coletando as múltiplas respostas obtidas, conseguem inferir informações sobre o modelo com base nas probabilidades do vocabulário. Diferentes modos de ataques podem ser feitos, seja injetando conteúdo malicioso direto no prompt ou camuflado em conteúdos externos (documentos, links, imagens ...), ou pode ocorrer a tentativa de sobrescrever configurações ou acessar parâmetros do sistema (ex: alterar prompt de instruções, controlar temperatura ...). Outra maneira possível de analisar as múltiplas respostas obtidas é com "engenharia reversa", em que atacantes conseguem inferir a arquitetura do modelo ou dados de treino que foram utilizados apenas com base na saída da API. Também pode-se explorar falhas de sistema, como tokens excedendo limites permitidos, informações escondidas e scripts que escalam privilégios ... além de outras abordagens.

Pode-se comentar também sobre mecanismos de defesa nesse caso que acabam tentando coibir ataques, seja por exemplo promovendo formas de não ser possível sobrescrever instruções do sistema, filtrando conteúdo de entrada e de saída além de proibir (via treinamento e instruções, além de bloquear pelo código) a capacidade de executar scripts maliciosos. Vale também melhorar a arquitetura de sistema do ponto de vista das requisições feitas, evitando ataques dos, além de bloquear atividades de usuários suspeitos.

A ideia geral é abordar de maneira aberta o tema do seminário e entender as possibilidades da área.

Enunciado:

Qual a necessidade de nos preocuparmos sobre segurança e privacidade relacionado aos modelos de linguagem?

Resposta:

Com o passar dos anos, os modelos crescem cada vez mais e obviamente precisam de mais dados. Dessa forma, com o uso cada vez mais crescente, essas brechas podem ser mais expostas prejudicando a vida das pessoas, seja propagando informações privadas ou então sendo usadas para realizar ações maliciosas. As ameaças são de diversas frentes, seja

atacantes que exploram vulnerabilidades dos sistemas de diferentes maneiras (ataques de prompt, injeção de conteúdo malicioso, negação de serviço, uso dos modelos para executar ações ilegais ...), seja também por conta das empresas que constroem esses modelos, que precisam de cada vez mais dados de treinamento e acabam obtendo de fontes ilegais. A privacidade de dados é muito relevante e vai se agravar ao longo do tempo pensando que cada vez temos mais conteúdo gerado por IA, e sem o devido controle, diversas informações falsas, que violam os direitos humanos e que quebram propriedades intelectuais de artistas vão estar em maior circulação, podendo modificar e danificar a vida das pessoas em larga escala. Assim mesmo que os sistemas tenham mecanismos de proteção, estes devem acima de tudo focar em evitar que os modelos possam realizar ações danosas à população e devem estar sempre atualizados em relação a novos padrões de ataque e ameaças de segurança.

Enunciado:

Você foi contratado para fazer a parte de segurança de uma empresa de LLMs, e, ao chegar na empresa, te apresentam o desafio atual da empresa. Atualmente, a cada modelo que nasce, gasta aproximadamente 3 dias para que também se encontre o Jailbreak do mesmo modelo. A empresa não tem qualquer tipo de proteção, portanto, faça uma proposta de como solucionar, ou pelo menos mitigar, o problema

Resposta:

A ideia da resposta é falar que para mitigar o problema de jailbreaks rápidos, uma estratégia eficaz e robusta é a implementação de um Modelo Supervisor, também chamado de *Guardrail Model* ou *Moderation Layer*, que opera em cascata antes e/ou depois do Large Language Model principal. O funcionamento básico consiste em treinar ou ajustar um modelo auxiliar menor para atuar como um classificador de risco. O processo de mitigação se daria em três etapas principais: primeiro, o Pré-filtro (*Input Moderation*), onde o Modelo Supervisor analisa o prompt do usuário antes que ele chegue ao LLM principal, identificando padrões de linguagem, frases-chave ou técnicas de ofuscação que são característicos de tentativas de *jailbreak*. Se o prompt for classificado como suspeito ou malicioso, a requisição é bloqueada ou retorna uma mensagem de erro. Em seguida, o Pós-filtro (*Output Moderation*), em que, após o LLM principal gerar uma resposta, o Modelo Supervisor a inspeciona, verificando se a saída contém conteúdo proibido, discurso de ódio ou instruções para atividades ilegais. Se a resposta violar as políticas de segurança, ela é substituída por uma resposta segura ou censurada. Por fim, o Reforço Contínuo e Defesa em Profundidade é crucial: a eficácia do Supervisor é mantida através do re-treinamento constante utilizando os *jailbreaks* reais encontrados, transformando o ciclo de ataque em um ciclo de defesa e aprendizado. Outras respostas como Pré treinamento do modelo ou fine tuning para evitar esse tipo de situação também estariam parcialmente certas, embora não seja a estratégia mais eficaz abordada pelo mercado.